

Devoir surveillé

Durée 2h
(Corrections)

Aucun document, outil de calcul ou de communication autorisé. Sauf indications contraires, la gestion d'erreur **ne doit pas** être réalisée et les inclusions ne doivent pas être spécifiées.
Toute réponse doit être justifiée.

Exercice 1 (Switch applicatif - les tubes nommés)

Nous souhaitons réaliser une application, que nous appellerons *switch*, permettant à $n > 1$ applications indépendantes de communiquer entre elles. Le *switch* prend en argument le nombre n puis crée ensuite $2 \times n$ tubes nommés qui seront utilisés pour les communications avec les n applications. Chacune est identifiée par un numéro de 1 à n . Quand elle désire envoyer des données (quelconques, de tailles variables) à une autre application, elle les envoie au *switch* qui les transfère.

Première partie (4,5pts)

1. Aurait-il été possible d'utiliser des tubes anonymes ? Si oui, expliquer votre solution. Sinon, expliquez pourquoi.

Solution : (0,5pts) *Ce sont des applications indépendantes, donc c'est impossible.*

2. Est-il possible qu'il y ait plus de n applications qui se "connectent" au *switch* ? Si oui, expliquez ce que cela entraînerait et proposez une solution pour l'éviter.

Solution : (0,5pts) *Plusieurs applications peuvent ouvrir le même tube. Du coup, elles pourraient lire chacune les messages de l'autre. Plusieurs solutions mais sans doute la plus simple : les sémaphores. Mais une file de messages est possible. Éviter les solutions avec les tubes où les processus lisent puis remettent dans le tube...*

3. Aurait-il été possible d'utiliser seulement un tube par application pour les communications avec le *switch* ? Expliquez.

Solution : (0,5pts) *Les applications doivent à la fois envoyer des informations puis en recevoir. Si une application envoie un message et attend une réponse, elle peut lire son propre message. Sinon, il aurait fallu utiliser des sémaphores.*

4. Indiquez quelles données (type, taille, taille maximum) sont envoyées dans les tubes.

Solution : (0,5pts) *Chaque application doit envoyer le numéro de l'application destinataire (un `int`), la taille des données (un `size_t`) et les données (un tableau de `char`, la taille maximum n'est pas un problème car il n'y a pas de concurrence).*

5. Donnez la portion de code permettant de lire les données depuis un tube nommé supposé ouvert. Vous préciserez la déclaration des variables utilisées et/ou des variables supposées initialisées.

Solution : (2pts) *Voici une solution :*

```
int fd; /* Descripteur de fichier du tube */
int num;
size_t taille;
unsigned char *data;

read(fd, &num, sizeof(int));
read(fd, &taille, sizeof(size_t));
data = (unsigned char*) malloc(sizeof(unsigned char)*taille);
read(fd, data, sizeof(unsigned char)*taille);
```

6. Une fois qu'il a créé tous les tubes, le *switch* peut-il tous les ouvrir en attendant que les applications soient exécutées ? Expliquez.

Solution : (0,5pts) L'ouverture est bloquante. Du coup, il ne peut ouvrir un tube tant qu'il n'y a pas une application en face.

Deuxième partie (3,5 pts)

Nous supposons pour simplifier qu'une fois le *switch* exécuté, toutes les applications se "connectent" avant de commencer la moindre communication. Pour permettre les communications en parallèle, nous souhaitons que le *switch* crée des processus fils qui se focalisent chacun sur la réception des données d'une seule application. Le fils envoie ensuite les données vers la bonne application.

1. Expliquez tout ce que doit faire un fils du *switch*.

Solution : (0,5pts) Dans un premier temps, il doit ouvrir tous les tubes en lecture/écriture. Puis, il se met en attente de données sur son tube. Dès qu'il reçoit des données, il les envoie dans le tube correspondant.

2. Quels problèmes de concurrence cela entraîne-t-il ? Comment les résoudre ?

Solution : (0,5pts) Si plusieurs fils écrivent en même temps dans le même tube, il peut y avoir des problèmes de concurrence si la taille des données est trop grande. La solution : limiter la taille des données.

3. Nous supposons que la fonction *fils* correspond à l'exécution d'un fils.

- (a) Donnez la signature de la fonction et expliquez les paramètres nécessaires.

Solution : (0,5 pts) Chaque fils a besoin de son numéro, ainsi que du nombre *n*.

- (b) Donnez le code du *main* du *switch* qui doit créer les tubes et les fils. Il attend ensuite la fin de chacun d'eux.

Solution : (2 pts)

```
int n = atoi(argv[1]);
char buffer[256];
int i;

for(i = 0; i < n; i++) {
    sprintf(buffer, "tube_%d", i);
    mkfifo(buffer, );
}

for(i = 0; i < n; i++) {
    if(fork() == 0)
        fils(i, n);
}

for(i = 0; i < n; i++)
    wait(NULL);

return EXIT_SUCCESS;
```

Exercice 2 (Application de log - les fichiers et les signaux - 9,5 pts)

Nous souhaitons développer une application de log que nous appellerons le *logger*. Celui-ci se met en attente de la réception de signaux SIGRTMIN envoyés par d'autres applications (que nous appellerons les *clients*).

Dès que le *logger* reçoit un tel signal, il crée un tube nommé dont le nom est *tube_PID.bin* où "PID" est le PID du *client*. Il se met alors en attente d'un message envoyé par le *client* (une chaîne de caractères de longueur variable). Le *logger* stocke l'ensemble des messages reçus dans des fichiers binaires, un pour chaque *client*. Chaque fichier est nommé *PID.bin* où "PID" est le PID du client. Comme les messages peuvent être de tailles différentes, nous souhaitons utiliser une table de positions stockée au début du fichier. Cette table contient un nombre fixe d'entrées (nous utiliserons la constante *MAX_ENTREES*), chacune contenant l'horodatage (un *timestamp* Unix) et la position du message dans le fichier.

1°) Pourquoi est-il préférable que la table possède un nombre fixe d'entrées ? Expliquez comment faire pour stocker plus de `MAX_ENTREES` entrées dans le fichier.

Solution : [1 pt] La table est stockée en début du fichier. Du coup, si sa taille était variable, il faudrait constamment décaler les entrées lors de l'ajout de nouvelles entrées. Pour gérer un nombre de messages plus grand, on utilise la première table pour stocker les premières entrées. Lorsqu'on arrive au nombre d'entrées maximum, on utilise une autre table stockée à la suite des messages.

2°) Proposez une structure en C pour les messages que vous nommerez `message_t`. Un message n'a pas de taille maximale.

Solution : [0,5 pt] Par exemple :

```
typedef struct {
    size_t taille;
    char *message;
} message_t;
```

3°) Donnez les structures en C de la table (nommée `table_t`) et d'une entrée (nommée `entree_t`).

Solution : [1 pt] Par exemple :

```
typedef struct {
    time_t horodatage;
    off_t position;
} entree_t;

typedef struct {
    entree_t [MAX_ENTREES];
    off_t suivante;
} table_t;
```

4°) Donnez le code de la fonction `int ouvrir_fichier(pid_t pid)`. Si le fichier nommé `X.bin` (où `X` est le PID) n'existe pas, il est créé et préparé : il contient une table d'entrées vide. Si le fichier existe déjà, il est simplement ouvert. Dans les deux cas, la fonction retourne le descripteur du fichier correspondant.

Solution : [2 pt] Par exemple :

```
int ouvrir_fichier(pid_t pid) {
    int fd;
    char tampon[25];
    sprintf(tampon, "%d.bin", pid);

    if((fd = open(tampon, O_CREAT | O_EXCL | O_RDWR, S_IRUSR | S_IWUSR)) == -1) {
        if(errno == EEXIST) {
            /* Le fichier existe déjà */
            fd = open(tampon, O_RDWR);
        }
        else {
            /* Le fichier n'existe pas */
            table_t table;
            memcpy(&table, '\0', sizeof(table_t));
            write(fd, &table, sizeof(table_t));
        }
    }
    return fd;
}
```

5°) Nous supposons l'existence de la fonction `off_t rechercher_vide(int fd)` qui retourne la position dans le fichier de la prochaine entrée de table vide. Expliquez le fonctionnement de cette fonction.

Solution : [1 pt] La fonction commence par se positionner au début du fichier. Elle lit la table et la parcourt. Si une entrée est vide, elle retourne la position. Sinon, elle déplace le curseur vers la prochaine table et la lit. Elle recommence la procédure jusqu'à trouver une place ou que la dernière table soit pleine et ne pointe vers aucune autre table. Dans ce cas, on se place à la fin du fichier et on stocke une nouvelle table vide. On n'oublie ensuite de modifier l'avant-dernière table pour qu'elle pointe sur la nouvelle table.

6°) Donnez le code de la procédure `void message_sauvegarder(message_t msg)` qui stocke le message passé en paramètre dans le fichier correspondant (utilisez les fonctions précédentes).

Solution : [2,5 pt]

```
void message_sauvegarder(message_t msg) {
    int fd = creer_fichier(msg.pid); /* Création/ouverture du fichier */
    off_t position = rechercher_vide(fd); /* Recherche/création d'une entrée vide */
    entree_t entree;
    int taille = strlen(msg.message) + 1;

    entree.position = lseek(fd, 0L, SEEK_END); /* On se place à la fin */
    entree.horodatage = msg.horodatage;
    write(fd, &taille, sizeof(int)); /* On écrit la taille du message */
    write(fd, msg.message, taille); /* On écrit le message */
    lseek(fd, entree.position, SEEK_SET); /* On se replace sur l'entrée */
    write(fd, &entree, sizeof(entree_t)); /* On sauvegarde la nouvelle entrée */
    close(fd);
}
```

7°) Que se passe-t-il lors de l'ajout si le nombre de messages devient trop grand dans le fichier ?

Solution : [0,5 pt] Le coût de la recherche devient très important. Il faut en effet lire toutes les tables jusqu'à trouver une entrée libre. Donc, beaucoup de lectures et de déplacements.

8°) Proposez une solution pour corriger ce(s) problème(s).

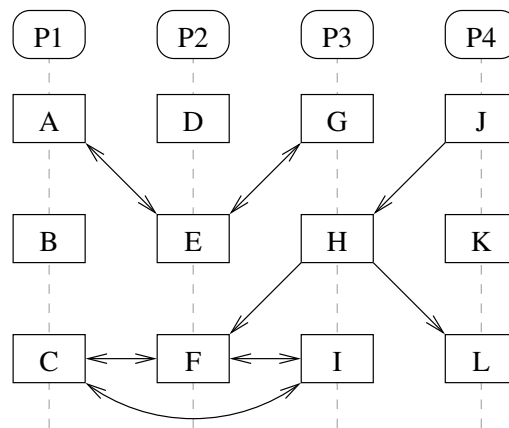
Solution : [0,5 pt] Une solution consisterait à mémoriser au début du fichier la position de la dernière table. Ainsi, lors de l'ajout d'une nouvelle entrée, on recherche uniquement dans cette table ou on en ajoute une nouvelle.

9°) Discutez sur le choix d'avoir utilisé les noms de fichiers `PID.bin` où "PID" est le PID du client qui a envoyé le message.

Solution : [0,5 pt] Au redémarrage du système ou si le système s'exécute depuis un long moment, les PID risquent d'être identiques pour des processus différents. Ainsi, les messages de processus différents seraient stockés dans le même fichier.

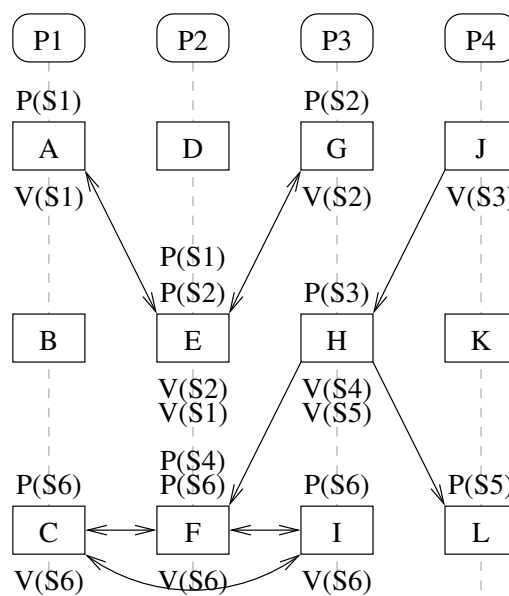
Exercice 3 (Sémaphores de Dijkstra - 3,5 pts)

Soit le diagramme de précedence suivant, représentant 4 processus P1, P2, P3 et P4 contenant chacun 3 blocs s'exécutant les uns après les autres :



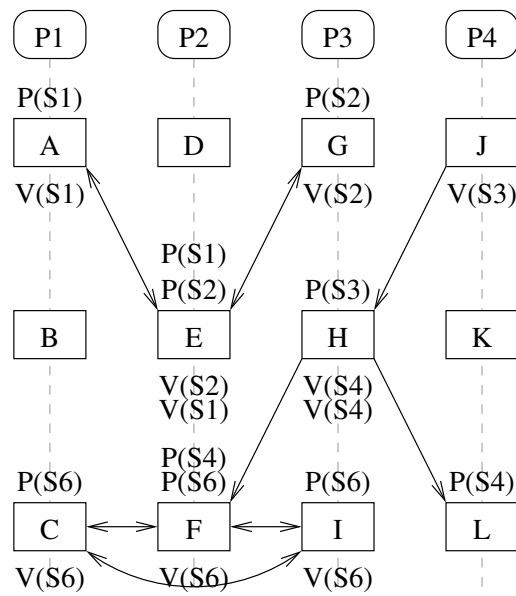
1°) Proposer une solution à base de sémaphores pour résoudre ces contraintes (1 sémaphore pour chaque cas).

Solution : (2pts) Avec $init(S1,1)$, $init(S2,1)$, $init(S3,0)$, $init(S4,0)$, $init(S5,0)$, $init(S6,1)$



2°) Proposez une solution pour réduire le nombre de sémaphores.

Solution : (0.5pts) Le sémaphore 5 peut être supprimé. Avec $init(S1,1)$, $init(S2,1)$, $init(S3,0)$, $init(S4,0)$, $init(S6,1)$



3°) Expliquez pourquoi la solution proposée pour résoudre les contraintes entre A, E et G n'est pas satisfaisante.

Solution : (1pts) Supposons que le processus $P3$ désire entrer en premier en exclusion mutuelle sur G . Il réalise $P(S2)$ en premier puis commence à exécuter son code. Le processus $P2$ souhaite également entrer en section critique sur E . Il existe $P(S1)$ et reste bloqué. Puis le processus $P1$ souhaite entrer en section critique sur A . Il est bloqué alors que le bloc E n'est pas en cours d'exécution.